

REST 練習 7：PUT/DELETE 與統一錯誤格式

Lecture

到 rs06 為止，四個 HTTP 動詞已經出現了三個：**GET**（查詢）、**POST**（建立）。這題補齊剩下兩個會改變伺服器狀態的動詞：**PUT**（更新）跟 **DELETE**（刪除）。

PUT 跟 **POST** 最關鍵的差異在「呼叫端是否已經知道資源的身分」：**POST /flights** 是「在集合底下建立一新筆新的，新資源的身分由伺服器決定」；**PUT /flights/{carrid}/{connid}/{fldate}** 是「更新已經知道身分的這一筆」——URI 本身就是這筆資源的識別碼，呼叫端不是在猜、是在指名道姓地說「就是這一筆」。也因為這樣，**PUT** 語意上是冪等的（**idempotent**）：同樣的 **PUT** 請求送一次跟送十次，最終資源的狀態應該完全一樣（都是「這筆資源的內容變成這樣」），不會像 **POST** 那樣每送一次就多一筆。**DELETE** 也是冪等的：刪一次跟刪十次，資源最終都是「不存在」這個狀態（雖然第二次之後系統可能回 404 而不是 204，但資源實際的存在狀態不會因為多刪幾次而改變）。

成功的狀態碼這題也出現兩個新面孔：**PUT** 成功回 **200 OK**（跟 **GET** 一樣，因為呼叫端通常想看到更新後的內容）；**DELETE** 成功回 **204 No Content**——這個狀態碼特別的地方在於它明確表示「成功了，但沒有東西要回給你」，所以 Response Body 應該是空的，這跟 **200/201** 都會帶一個有意義的 Body 不一樣。之所以要有這個專門的狀態碼，是因為「刪除成功」這件事本身沒有「刪掉的東西」可以回顯——回傳被刪掉的資料意義不大（呼叫端刪它就是因為不想要它了），回傳空物件 `{}` 又容易讓人誤會是不是漏東西沒填，**204** 用狀態碼本身就講清楚「你要的動作做完了，沒有內容」。

這題另一個重點是**統一錯誤格式**。rs01~rs06 每次遇到錯誤都是 **RAISE EXCEPTION TYPE cx_rest_resource_exception**——這個框架內建的例外類別只能帶「狀態碼」跟「request method」，沒有欄位可以放「機器可讀的錯誤代碼」或「給人看的說明訊息」，呼叫端拿到的 Response Body 內容完全由框架自己決定格式，Resource 類別無法客製化。這題改用自訂的業務例外類別（**ZCX_RS07_FLIGHT_ERROR**，作法跟 op09「自訂例外」同一套：繼承 **CX_STATIC_CHECK**，帶自己的屬性），讓每個錯誤情境除了 HTTP 狀態碼之外，還能附上 **errorCode**（例如 **FLIGHT_NOT_FOUND**、**VALIDATION_FAILED**，給程式判斷用）跟 **message**（給人看的說明），然後在 Resource 類別裡攔截這個例外、統一轉成同一種 **JSON** 結構回應：
`{"errorCode": "...", "message": "..."}。`不管是 400、404 還是這題新增的驗證錯誤，呼叫端拿到的錯誤 Body 格式永遠長得一樣——這對一個真的要給別人呼叫的 API 非常重要：呼叫端的錯誤處理程式碼只需要寫一套「解析 **errorCode/message**」的邏輯，不用針對每個狀態碼各寫一套解析規則。

最後，「攔截例外、避免變成 Dump」是這題想補的最後一塊：前六題的程式碼裡，任何沒有被明確 **RAISE**，卻在執行過程中意外發生的技術性錯誤（例如型別轉換失敗），如果沒有被攔截，會直接變成 ABAP 系統 Dump（**ST22** 看得到的那種），呼叫端收到的不是一個像樣的 HTTP 錯誤回應，而是連線中斷或一個充滿系統內部細節的錯誤頁面——這對外部 API 呼叫端來說既不友善、也可能洩漏系統內部資訊。這題在每個方法的最外層都加了 **CATCH cx_root**，把任何「非預期」的技術性例外也包裝成同樣的統一 **JSON** 錯誤格式（**500 Internal Server Error + errorCode: INTERNAL_ERROR**），確保呼叫端永遠拿到的是一個結構一致的 **JSON**，不會有 Dump 這種「開天窗」的狀況。

學習目標

- 分清楚 **PUT**（更新已知身分的資源，冪等）跟 **POST**（建立新資源，非冪等）的語意差異；分清楚 **DELETE** 成功要回 **204 No Content**（無 Body）而不是 **200**

- 學會設計一個自訂業務例外類別取代框架內建的 `CX_REST_RESOURCE_EXCEPTION`：
`ZCX_RS07_FLIGHT_ERROR` 除了狀態碼，還帶 `errorCode`（機器可讀）與 `message`（人類可讀），比 `op09` 單純帶一個 `carrid` 屬性更進一步，是專門為了「統一轉成 JSON error body」設計的
- 學會在 Resource 類別裡用 `TRY...CATCH` 攔截自訂例外，集中在一個私有方法（`render_error`）裡把例外轉成一致的 JSON 結構，讓 `GET/PUT/DELETE` 三個方法共用同一套錯誤回應邏輯，不用各自重複組 JSON
- 認識防禦式攔截的必要性：除了攔截自己定義的業務例外，外層再加一層 `CATCH cx_root` 攔截任何非預期的技術性例外，包裝成 `500` 回應，確保呼叫端不會遇到系統 Dump
- 複習並修正 `rs06` 教過的地雷：字串樣板 `|{ }|` 直接內插 `DATS` 型別的值會套用使用者日期格式（可能出現點號），要組進錯誤訊息前得先用 `CONV string( ... )` 轉換——這題在寫 `read_flight` 的 404 訊息時第一版就真的踩到這個雷，寫出 `|...{ is_keys-fldate }|`，修正成 `|...{ CONV string( is_keys-fldate ) }|` 才正確（見下方「團隊實務備註」）

為什麼要統一錯誤格式、為什麼要多一層 CATCH cx_root

如果每個錯誤情境都各自組不同格式的 Response Body（有的回字串、有的回框架預設的空 Body、有的回自訂 JSON），呼叫端的錯誤處理程式碼會變得非常破碎：每一種狀態碼都要另外寫一段解析邏輯，甚至要先試著解析 JSON 失敗了才知道這次錯誤沒有 Body。統一錯誤格式的價值在於：呼叫端只要寫一次「檢查狀態碼是否為 **2xx**，不是的話解析 `errorCode/message`」的邏輯，就能處理這個 API 所有的錯誤情境，不用每加一個新的錯誤案例就要呼叫端跟著改程式碼。

`CATCH cx_root` 這道防線要處理的是「你沒預料到的錯」：業務邏輯裡明確檢查過的錯誤（缺欄位、查無資料）都已經被 `ZCX_RS07_FLIGHT_ERROR` 攔下來了，但程式難免有沒設想到的情境（型別轉換出乎意料地失敗、底層函式丟出跟你預期不同的技術性例外），這類錯誤如果沒被攔截，`ABAP` 執行期會直接產生系統 Dump——對一個 REST API 來說，Dump 意味著呼叫端收到的可能是連線中斷、或是一個充滿系統路徑與內部變數的錯誤頁面，這在正式環境是不能接受的（除了不友善，還可能洩漏系統內部資訊給不該看到的人）。多包一層 `CATCH cx_root`，把這類意外也轉成同一種 JSON 結構（`500 + INTERNAL_ERROR`），是防禦式程式設計的基本手法：明確預期的錯誤交給業務例外處理，剩下所有「你沒想到的」交給最外層的安全網，兩者合起來才能保證呼叫端永遠拿到一致、可預期的回應格式。

事前準備

ADT 端物件已由課程準備好（`$TMP`）：

- `ZCX_RS07_FLIGHT_ERROR`——自訂業務例外，繼承 `CX_STATIC_CHECK`，帶 `mv_status_code`（HTTP 狀態碼）、`mv_error_code`（機器可讀錯誤代碼）、`mv_message`（訊息）
- `ZCL_RS07_APP`——Application Class，router 掛一個資源 `/flights/{carrid}/{connid}/{fldate}`；覆寫 `HANDLE_CSRF_TOKEN` 跳過 CSRF 檢查（比照 `rs06`，供 `SPROX_HTTP_REQUEST` 測試用）
- `ZCL_RS07_FLIGHT`——同一個資源覆寫 `GET/PUT/DELETE`，共用私有方法處理路徑參數解析、查詢、成功/錯誤回應組裝

題目需求（對照已建好的答案物件）

1. `ZCX_RS07_FLIGHT_ERROR`：比照 `op09` `ZCX_0009_NO_FLIGHT` 的 `mv_/iv_` 命名慣例，多帶兩個屬性：

```
``abap CLASS zcx_rs07_flight_error DEFINITION PUBLIC INHERITING FROM cx_static_check FINAL CREATE PUBLIC. PUBLIC SECTION. DATA mv_status_code TYPE i READ-ONLY. DATA mv_error_code TYPE string READ-ONLY. DATA mv_message TYPE string READ-ONLY.
```

METHODS: constructor IMPORTING iv_status_code TYPE i iv_error_code TYPE string iv_message TYPE string
previous LIKE previous OPTIONAL, get_text REDEFINITION. ENDCLASS.

CLASS zcx_rs07_flight_error IMPLEMENTATION. METHOD constructor ##ADT_SUPPRESS_GENERATION. super->constructor(previous = previous). mv_status_code = iv_status_code. mv_error_code = iv_error_code.
mv_message = iv_message. ENDMETHOD.

METHOD get_text. result = mv_message. ENDMETHOD. ENDCLASS. ``

1. **ZCL_RS07_APP** : 路由掛法跟 rs04 單筆查詢一樣 (同一個路徑樣板) , CSRF 跳過寫法比照 rs06 :

```
``abap METHOD if_rest_application~get_root_handler. DATA(lo_router) = NEW cl_rest_router(). lo_router->attach( iv_template = '/flights/{carrid}/{connid}/{fldate}' iv_handler_class = 'ZCL_RS07_FLIGHT' ).  
ro_root_handler = lo_router. ENDMETHOD.
```

METHOD handle_csrf_token.

- 刻意留空、不呼叫 SUPER-> , 跳過 CSRF Token 檢查 (比照 rs06)

ENDMETHOD. ``

1. **ZCL_RS07_FLIGHT** : GET/PUT/DELETE 共用幾個私有方法 :

- **read_keys()** : 解析 **carrid/connid/fldate** 三個路徑參數 , **fldate** 格式不合法 (不是 8 碼數字) 就 **RAISE zcx_rs07_flight_error (400/INVALID_DATE)**
- **read_flight(is_keys)** : 用三個鍵值查 **SFLIGHT** , 查無資料就 **RAISE zcx_rs07_flight_error (404/FLIGHT_NOT_FOUND)**
- **render_flight(is_flight)** : 把查到的航班資料序列化成 JSON 回應 (跟前幾題一路用的 **/UI2/CL_JSON=>SERIALIZE** 一樣)
- **render_error(ix_error)** : 把 **ZCX_RS07_FLIGHT_ERROR** 的 **mv_error_code/mv_message** 組成 **{ "errorCode": "...", "message": "..." }** , 設定對應的 HTTP 狀態碼
- **wrap_error(ix_error)** : 把任何 **CX_ROOT** (非預期的技術性例外) 包裝成 **zcx_rs07_flight_error (500/INTERNAL_ERROR)** , 交給 **render_error** 統一處理

read_keys 這一版是修正過的版本 , 第一版曾經誤以為 **CONV dats(...)** 對非數字字串轉型失敗會拋出可攔截的例外 (rs04/rs05 都留了這個假設當「已知限制」) , 實際掛 **SICF** 測試 **/flights/LH/400/abcd** 才發現不是這樣——**DATS** 底層是字元型別 , **CONV dats('abcd')** 不會報錯 , 只是把不合法的內容原樣塞進去 , 導致後面查詢資料庫時單純「查無資料」變成 **404** , 而不是預期中的 **400** 。正確做法是**自己驗證字元組成與長度** , **不要依賴轉型失敗** :

```
``abap METHOD read_keys. rs_keys-carrid = mo_request->get_uri_attribute( 'carrid' ). rs_keys-connid =  
mo_request->get_uri_attribute( 'connid' ).
```

```
DATA(lv_fldate_raw) = mo_request->get_uri_attribute( 'fldate' ).
```

- **CONV dats(...)** 對非數字字串不會拋出例外 (**DATS** 底層是字元型別 , 不做內容檢查) ,
- 必須自己驗證字元組成與長度 , 不能依賴轉型失敗來偵測格式錯誤

```
IF strlen( lv_fldate_raw ) <> 8 OR lv_fldate_raw CN '0123456789'. RAISE EXCEPTION TYPE zcx_rs07_flight_error  
EXPORTING iv_status_code = cl_rest_status_code=>gc_client_error_bad_request iv_error_code =  
'INVALID_DATE' iv_message = |fldate 格式不正確 , 必須是 8 碼數字 ( YYYYMMDD ) | . ENDIF.
```

```
rs_keys-fldate = lv_fldate_raw. ENDMETHOD. ``
```

CO/CN 是 ABAP 字元集合比較運算子：lv_fldate_raw CN '0123456789' 意思是「lv_fldate_raw 裡存在不屬於 0123456789 這個字元集合的字元」（CN = contains not only），配合 strlen(...) <> 8 就能同時擋掉「太短/太長」跟「含非數字字元」兩種不合法輸入，不需要依賴任何轉型例外。

GET：

```
``abap METHOD if_rest_resource~get. TRY. DATA(ls_keys) = read_keys( ). DATA(ls_flight) =
read_flight( ls_keys ). render_flight( ls_flight ). CATCH zcx_rs07_flight_error INTO
DATA(lx_error). render_error( lx_error ). CATCH cx_root INTO DATA(lx_unexpected).
render_error( wrap_error( lx_unexpected ) ). ENDTRY. ENDMETHOD. ``
```

PUT：body 只需要 price/currency 兩個欄位（主鍵已經在 URL 裡了，不用重複帶），缺任一個回 400：

```
``abap METHOD if_rest_resource~put. TRY. DATA(ls_keys) = read_keys( ). read_flight( ls_keys )."確認資源存在，
查無資料會直接拋出 404
```

```
DATA(lv_body) = io_entity->get_string_data( ).
```

```
TYPES: BEGIN OF ty_update, price TYPE s_price, currency TYPE s_currcode, END OF ty_update. DATA ls_update
TYPE ty_update.
```

```
/ui2/cl_json=>deserialize( EXPORTING json = lv_body pretty_name = /ui2/cl_json=>pretty_mode-camel_case
CHANGING data = ls_update ).
```

```
IF ls_update-price IS INITIAL OR ls_update-currency IS INITIAL. RAISE EXCEPTION TYPE zcx_rs07_flight_error
EXPORTING iv_status_code = cl_rest_status_code=>gc_client_error_bad_request iv_error_code =
'VALIDATION_FAILED' iv_message = |price 與 currency 都必填|. ENDIF.
```

```
UPDATE sflight SET price = ls_update-price currency = ls_update-currency WHERE carrid = ls_keys-carrid AND
connid = ls_keys-connid AND fldate = ls_keys-fldate.
```

```
DATA(ls_updated) = read_flight( ls_keys ). render_flight( ls_updated ). CATCH zcx_rs07_flight_error INTO
DATA(lx_error). render_error( lx_error ). CATCH cx_root INTO DATA(lx_unexpected). render_error( wrap_error(
lx_unexpected ) ). ENDTRY. ENDMETHOD. ``
```

DELETE：IF_REST_RESOURCE~DELETE 沒有 IO_ENTITY 參數（介面定義裡 DELETE 不吃任何 Request Body，符合語意——刪除不需要帶內容）：

```
``abap METHOD if_rest_resource~delete. TRY. DATA(ls_keys) = read_keys( ). read_flight( ls_keys ).
```

```
DELETE FROM sflight WHERE carrid = ls_keys-carrid AND connid = ls_keys-connid AND fldate = ls_keys-fldate.
```

```
mo_response->set_status( cl_rest_status_code=>gc_success_no_content ). CATCH zcx_rs07_flight_error INTO
DATA(lx_error). render_error( lx_error ). CATCH cx_root INTO DATA(lx_unexpected). render_error( wrap_error(
lx_unexpected ) ). ENDTRY. ENDMETHOD. ``
```

幾個實作細節：

- PUT/DELETE 都先呼叫 read_flight(ls_keys) 確認資源存在，查無資料會由 read_flight 直接拋出 404——不用在 PUT/DELETE 裡另外寫一次「查無資料」的檢查，這正是把驗證邏輯集中在

共用私有方法的好處

- **PUT** 成功後又呼叫一次 **read_flight** 把更新後的資料讀回來再序列化回應，不是直接拿 **ls_update** 組一個殘缺的 JSON (**ls_update** 只有 **price/currency**，沒有 **carrid/connid/fldate**) ——讓呼叫端拿到的 **PUT** 回應跟 **GET** 長得一模一樣 (完整的航班資料)，是比較好的 REST 設計慣例
- **DELETE** 成功後只呼叫 **mo_response->set_status(... gc_success_no_content)**，完全不呼叫 **mo_response->create_entity()**——**204** 語意上就是「沒有內容」，不建立 entity，Response Body 才會真的是空的
- **204** 沒有內容可看，怎麼確認真的刪除成功？——用同一個 URL 再 **GET** 一次，如果回 **404** 就代表資料真的不在了 (見下方 SICF 測試步驟第 5 點)

SICF 掛載步驟 (比照 rs01~rs06)

沿用既有的 **/sap/bc/zrest_training** 分類節點，這題只要在其底下再加一個子節點：

1. SICF 交易碼，在 **zrest_training** node 上右鍵 → **New Sub-Element**，Service Name 填 **rs07**
2. Handler List 掛 **ZCL_RS07_APP**
3. Activate Service
4. 用 **SPROX_HTTP_REQUEST** 測試 (**PUT** 一樣需要自訂 Request Body，瀏覽器網址列做不到)。URL 一律填完整網址 (**http://<主機>:<port>/sap/bc/zrest_training/rs07/...?sap-client=130**)：

- **GET** 現況 (更新前)：**.../rs07/flights/LH/400/20190104**，HTTP Method **GET**，預期看到 **price** 是 **666.00**、**currency** 是 **EUR**
- **PUT** 成功更新 (**200**)：同一個 URL，HTTP Method **PUT**，Req. Body：

```
``json {"price":680.00,"currency":"EUR"}``
```

- **GET** 驗證更新結果：再 **GET** 同一個 URL，確認 **price** 變成 **680.00**
- **PUT** 缺必填欄位 (**400**)：同一個 URL，HTTP Method **PUT**，Req. Body 只給 **price**：

```
``json {"price":680.00}``
```

- **PUT/GET** 查無資源 (**404**)：**.../rs07/flights/LH/400/20991231** (這組鍵值資料庫裡不存在)
- **fldate** 格式錯誤 (**400**)：**.../rs07/flights/LH/400/abcd** (**fldate** 不是合法的 8 碼數字)
- **DELETE** 成功 (**204**)：**.../rs07/flights/AA/17/20260101** (這是 rs06 練習時建立的那筆資料，如果還在就會刪成功；如果之前已經手動清理過，這步驟會回 **404**，正好用來驗證「查無資源」的路徑，兩種結果都是合理的教學示範)，HTTP Method **DELETE**，不用填 Req. Body
- **DELETE** 後再驗證：對同一個 URL 再 **GET** 一次，確認回 **404**；如果對同一筆再 **DELETE** 一次，也會是 **404** (示範 **DELETE** 的冪等語意：刪過的東西再刪一次，資源「不存在」這個最終狀態沒有改變，只是狀態碼從 **204** 變成 **404**)

預期輸出 (範例)

GET .../flights/LH/400/20190104 (更新前)：HTTP 狀態碼 **200**

```
{"carrid":"LH","connid":400,"fldate":"2019-01-04","price":666.00,"currency":"EUR"}
```

PUT .../flights/LH/400/20190104，Body **{"price":680.00,"currency":"EUR"}**：HTTP 狀態碼 **200**


```
{"carrid":"LH","connid":400,"fldate":"2019-01-04","price":680.00,"currency":"EUR"}
```

PUT 缺 currency : HTTP 狀態碼 400

```
{"errorCode":"VALIDATION_FAILED","message":"price 與 currency 都必填"}
```

GET/PUT 查無資源 (.../flights/LH/400/20991231) : HTTP 狀態碼 404

```
{"errorCode":"FLIGHT_NOT_FOUND","message":"找不到航班 LH/0400/20991231"}
```

fldate 格式錯誤 (.../flights/LH/400/abcd) : HTTP 狀態碼 400

```
{"errorCode":"INVALID_DATE","message":"fldate 格式不正確，必須是 8 碼數字 (YYYYMMDD) "}
```

DELETE .../flights/AA/17/20260101 : HTTP 狀態碼 204 · Response Body 空白。

團隊實務備註

- 這題第一版程式碼真的踩到了 rs06 教過的地雷：read_flight 的 404 錯誤訊息一開始寫成 |找不到航班 { is_keys-carrid }/{ is_keys-connid }/{ is_keys-fldate }|，直接把 DATS 型別的 is_keys-fldate 放進字串樣板內插——這正是 rs06 筆記提醒過的「字串樣板對 DATS 型別會套用使用者日期顯示格式（可能出現點號）」問題，寫的時候一時疏忽又犯了同樣的錯，後來檢查程式碼時才發現並修正成 CONV string(is_keys-fldate)。這說明「知道一個坑」跟「寫程式時不再踩到」是兩件事，這類容易忽略的細節值得養成寫完之後對照檢查清單複查一次的習慣。
- connid 在錯誤訊息裡顯示成 0400（帶前導 0）：is_keys-connid 是 NUMC 型別，內部儲存就是補滿位數的字串，字串樣板內插 NUMC 不會像 DATS 那樣做特殊格式轉換，看到的就是內部值——這跟 rs04/rs06 提過的「/UI2/CL_JSON 序列化 NUMC 會自動去掉前導 0」是兩回事：/UI2/CL_JSON 序列化有自己的格式化規則，但字串樣板 |{ }| 內插一般型別（非 DATS）就是原始內部值，兩者不能混為一談。
- PUT 不驗證 carrid 存在性（不像 rs06 POST 會另外查 SCARR 驗證 carrid 是否為合法航空公司代碼）：因為 PUT 更新的是「已經確認存在的資源」（read_flight 已經驗證過鍵值對應到一筆真實的 SFLIGHT 記錄），carrid 既然能查到資料就代表當初建立時已經通過外鍵檢查，這裡不用重複驗證；PUT 的 Request Body（price/currency）跟主鍵完全無關，不會動到 carrid 有效性這件事。
- 204 No Content 不能用 SE16/SE16N 之外的方式「看到」刪除結果，只能用狀態碼跟後續 GET 間接確認：這是 204 這個狀態碼本身的設計意圖，不是這題程式碼的限制。

思考題

1. 如果呼叫端對同一筆資源在極短時間內送出兩個 DELETE 請求（類似 rs06 思考題 3 提到的 race condition），有沒有可能兩個請求都通過「資源存在」的 read_flight 檢查、但只有一個 DELETE FROM sflight 真的刪到資料？這種情況下第二個請求回應的狀態碼應該是什麼？（提示：DELETE FROM ... WHERE ... 即使沒有刪到任何資料列，sy-subrc 通常還是會是 0，這跟 INSERT 主鍵重複會讓 sy-subrc <> 0 的行為不一樣）
2. 這題的 wrap_error 把所有 CX_ROOT 都包成同一種 500/INTERNAL_ERROR，mv_message 裡直接帶入了原始例外的 get_text()——如果這個 API 是給外部客戶呼叫（不是內部教學環境），把系統內部例外的技術性訊息原封不動回給呼叫端，可能有什麼風險？正式環境通常會怎麼處理「要讓開發者看到詳細錯誤，又不能洩漏給外部呼叫端」這個兩難？

3. **PUT** 的語意是「用 **Body** 的內容整筆覆蓋這個資源」，但這題的 **PUT** 實作只更新 **price/currency** 兩個欄位，**SFLIGHT** 其他欄位（**PLANETYPE**、**SEATSMAX** 等）完全沒有被這次 **UPDATE** 語句碰到——這樣算是符合 **PUT** 「整筆覆蓋」的語意嗎？如果呼叫端以為 **PUT** 會把整筆資源換成 **Body** 給的內容（包括沒提到的欄位應該被清空或恢復預設值），會不會跟這題的實作產生認知落差？（提示：這種「只更新部分欄位」的語意在 **REST** 裡通常用哪個動詞表示更精確？）

答案

見 **zcx_rs07_flight_error.clas.abap**、**zcl_rs07_app.clas.abap**、**zcl_rs07_flight.clas.abap**（**SAP** 端物件 **ZCX_RS07_FLIGHT_ERROR** / **ZCL_RS07_APP** / **ZCL_RS07_FLIGHT**）。**SICF Service** 路徑 **/sap/bc/zrest_training/rs07**。